

Домашняя работа 2: генерация кода

Андрей Ворсин · ФКН ВШЭ · 3 октября 2026 года · итоговый отчет

Задача и ограничения

Обучить с нуля Transformer с энкодером вопроса и декодером Python-кода. Данные - только [выданный датасет](#): 11 838 003 пары question/code в train и 1 919 вопросов без ответов в test. Для итогового решения требуется сгенерировать все тестовые ответы не дольше 180 секунд на одной NVIDIA T4. Публичный pass@1 оценивает проверяющая система курса; тестовые ответы не опубликованы.

Данные и модель

Для проверки отложены первые 1 024 пары первого train-шарда. Эти пары не участвуют в обучении модели и токенизатора. Токенизатор byte-level BPE размера 8 192 обучен на остальных парах того же шарда. Модель T5-типа создана со случайными весами: 9 967 360 параметров, 4 слоя энкодера, 4 слоя декодера, размер скрытого состояния 256, 4 головы внимания. Ограничения длины: 128 токенов для вопроса и 192 для кода. Предобученные веса, дополнительные данные и искусственные таргеты не применялись.

Обучение

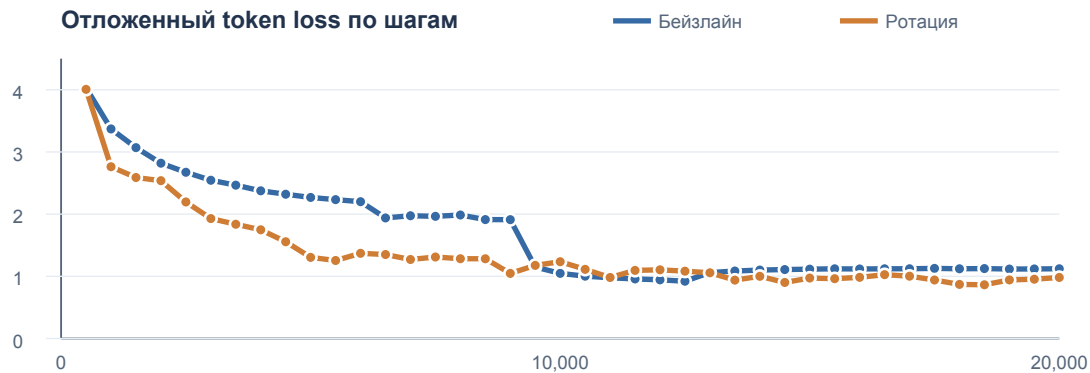
AdamW, скорость обучения 3×10^{-4} , warmup 500 обновлений, batch 16 × накопление градиента 4 = 64 примера на обновление; смешанная точность FP16, обрезка нормы градиента 1.0. Пробег - 20 000 обновлений на NVIDIA RTX PRO 6000 Blackwell Server Edition (Colab G4). Модель и состояние оптимизатора сохраняются каждые 500 шагов.

Проверенные эксперименты

Сначала один локальный шаг на Apple MPS проверил прохождение данных, обратный проход и сохранение модели (train loss 9.6595; held-out token loss 9.0907). Затем 10 шагов на T4 проверили совместимость среды и сохранение чекпоинта. Пилотный T4-прогон дошел до 4 000 шагов (loss 1.6469), но связь прервалась и итоговый чекпоинт не был получен. Этот пилот не участвует в выборе итоговой модели.

На корпоративной G4 проведены два запуска с одинаковой архитектурой, бюджетом обновлений и гиперпараметрами. Бейзлайн читает каждый перемешанный шард целиком; второй вариант переходит к следующему шарду после 1 000 мини-батчей. Гипотеза: при том же числе обновлений большее покрытие train-шардов улучшит обобщение на отложенных вопросах. Итоговый validation loss: 1.1213 против 0.9815. У бейзлайна был лучший записанный loss 0.9207 на шаге 12 500, однако его веса не сохранились: остался только последний чекпоинт. Для ответов выбрана модель: ротация train-шардов после 1 000 мини-батчей. Чекпоинт выбран по минимальному loss на первых 256 отложенных парах (шаг 18 500, loss 0.8636). Гипотеза о ротации поддержана меньшим минимальным loss; на отдельных 256 парах точных текстовых совпадений было 2 против 0 у бейзлайна.

Шаг	Loss на отложенных токенах
500	4.0064
1 000	2.7620
3 000	1.9287
5 000	1.3043
10 000	1.2348
18 500	0.8636
20 000	0.9815



Token loss измеряет предсказание следующего токена при известном правильном префиксе. Это не pass@1 и не проверка корректности выполненной программы.

Генерация и результат

Основная генерация жадная, пакетами по 64 вопроса. Вопросы сортируются по длине для уменьшения padding, затем ответы возвращаются в исходный порядок. Только синтаксически неверные ответы повторно генерируются с четырьмя лучами; один разделенный пробелом идентификатор исправлен, и результат проверен Python-парсером. Время измеряется на T4 с синхронизацией CUDA вокруг генерации всех тестовых строк, включая сортировку, токенизацию, повторные попытки и запись CSV. Результат: 1 919 ответов; время 39.09 с. Все сгенерированные ответы непустые и синтаксически корректные.

На 256 отложенных парах, не использованных для выбора чекпоинта, считаются доля непустого синтаксически корректного кода (0.9961), доля программ с функцией solve (0.9961) и точное совпадение с эталоном (0.0078). Эти числа - диагностика, не замена pass@1: правильную задачу можно решить разным кодом. Публичный pass@1: не измерено.

Использование агента

Codex помог прочитать условие и схему датасета, написать и проверить код обработки данных, архитектуру, цикл обучения, инференс и отчет. Он нашел ошибку передачи token_type_ids в generate во время локальной проверки; ошибка исправлена до GPU-прогона. Агент не создавал обучающие пары, дополнительные таргеты и эталонные ответы и не использовал чужие веса. Итоговые численные результаты взяты из выполненных запусков.

Ограничения и воспроизводимость

Формат посылки для Telegram-бота на момент работы еще не опубликован в условии. Код, токенизатор, Colab-ноутбуки и результаты лежат в форке [OsVox/hse-nlp](#). Для повторения нужны GPU для обучения, T4 для проверки лимита, доступ к датасету и запуск `hw2_train_colab.ipynb`, затем `hw2_inference_colab.ipynb`. Тестовый leaderboard и 4-балльный порог нельзя считать подтвержденными без публикации правил отправки и ответа проверяющей системы.

Источники: [условие курса](#); [описание датасета](#).